

Verslag Taak AD3

Naam: Isler Gabriel

Studentennummer: 0631137

E-mailadres: gabriel.isler@vub.be

Studierichting: Computerwetenschappen

Academiejaar: 2025-2026

1. Inleiding

Voor deze taak zijn we verplicht om conjunctieve selectiequery's toe te passen in het huidige database-ADT. Het betreft de functies `select-from/eq-and` voor sets van resultaten die binnen het centraal geheugen kunnen worden opgeslagen, en `select-from/eq-and+` voor resultaten die de capaciteit van het geheugen overschrijden en daarom externe opslag nodig hebben. De twee conjunctieve selectiequery's dienen effectief te verlopen.

Hiermee bedoelen we dat het aantal block-transfers zo laag mogelijk gehouden moet worden. Er moet dus rekening worden gehouden met het aantal beschikbare B+-tree-indices voor de gegeven attributen in de query.

2. Implementatie en keuzes

2.1 Implementatie

De wijzigingen zijn opgenomen in de `database.rkt` voor de zoeklogica en in het `schema.rkt` om de abstractie te waarborgen. Om conjunctieve zoekopdrachten efficiënt te behandelen, geven we de voorkeur aan het toepassen van indexen in plaats van een sequentiële zoekopdracht.

Het algoritme onderzoekt de conjunctieve voorwaarden (AND-clausules). Als er een B-tree index beschikbaar is voor ten minste één van de attributen in de query, wordt deze gekozen als het primaire toegangspad. Wanneer er verschillende indexen beschikbaar zijn, nemen we de eerste die we hebben gevonden.

We gebruiken de B-tree index (`btree:find!` gevolgd door `btree:set-current-to-next!`) om effectief de Record Identifiers (RCID's) te verzamelen die voldoen aan de geïndexeerde eisen. Met de gevonden RCID springen we via `table:current!` direct naar de juiste locatie in de tabel om het volledige record op te halen. Omdat de index slechts één voorwaarde dekt, filteren we de resultaten verder in het geheugen. Het predicaat `matches-conds?` controleert hierbij de resterende voorwaarden.

2.2 Performantie

De efficiëntie wordt vastgesteld aan de hand van het aantal diskblocktransfers, aangezien dit de belangrijkste factor is voor de prestaties van externe datastructuren.

Het sequentieel doorlopen vraagt om het inlezen van alle blokken in de tabel $O(N)$ blocks. Met de index kunnen we het aantal gelezen blokken beperken tot de hoogte van de B-tree $O(\log N)$.

De verwerking gebeurt stap voor stap, waarbij we voorkomen dat we tussenliggende lijsten in het geheugen plaatsen. We hebben de huidige pointers van de B-tree en tabel

ADT's. Dit zorgt ervoor dat de complexiteit gelijk blijft aan $O(1)$, wat ervoor zorgt dat de query ook uitvoerbaar blijft wanneer de dataset groter is dan het werkgeheugen dat beschikbaar is.

2.3 Uitbreiding Schema ADT

De functie `schema:description` is aan het schema ADT toegevoegd, zodat we de lijst met tabeleigenschappen, zoals namen en types, opnieuw kunnen opvragen. Dit is essentieel omdat de `+ query-variant` de functie voor het opstellen van een tabel gebruikt, die deze lijst als input vereist. Door deze functie te activeren, hoeft de database-module niet meteen in de ruwe bits en bytes van het schema-blok te kijken om te bekijken hoe de tabel eruitziet. Dit garandeert dat de technische gegevens over de opslag goed verborgen blijven binnen de ADT, zoals het hoort.

2.4 Select-from/eq-and+

De `select-from/eq-and+` functie is ontworpen om te voorkomen dat het werkgeheugen overstroomt wanneer een zoekopdracht veel resultaten oplevert. In plaats van al deze resultaten in een lange lijst te plaatsen, kopiëren we elk gevonden resultaat direct naar een nieuwe tabel op de harde schijf (via `tbl:insert!`). Dit zorgt ervoor dat het systeem goed functioneert, zelfs wanneer de hoeveelheid gegevens groter is dan wat er in het interne geheugen kan worden opgeslagen. Daarnaast blijft deze nieuwe tabel behouden, zodat we deze later weer kunnen inzetten voor toekomstige verzoeken.